Cairo University Faculty of Engineering Computer Engineering Department	CMPS 301 Fall 2026



Architecture Project

Objective

To design and implement a simple 6-stage pipelined processor, **Von-neumann (One memory for program and data)**.

Processor Stages:

1- Fetch Stage: Fetches the next instruction from the memory.
2- Decode Stage: Decodes instruction and releases control signals.
3- Execute-1 Stage: ALU operations.
4- Execute-2 Stage: Branch decision resolution, memory LDD/STD address calculation.
5- Memory Stage: For accessing memory in memory operations.
6- Write-Back Stage: For updating the register file.
The design should conform to the ISA specification described in the following sections.

Introduction

The processor in this project has a RISC-like instruction set architecture. There are eight 4-byte general purpose registers; R0, till R7. Another two special purpose registers, one works as a program counter (PC). The second is stack pointer (SP); and hence; points to the top of the stack. The initial value of SP is (2^12-1). The memory address space is 4K words (4096 × 32-bit) with a synchronous read (data appears one clock cycle after the address). The data bus is 32 bits.

When an interrupt occurs, the processor finishes the currently fetched instructions (instructions that have already entered the pipeline), then the address of the next instruction (in PC) is saved on top of the stack, and PC is loaded from address 1 of the memory. To return from an interrupt, an RTI instruction loads the PC from the top of stack, and the flow of the program resumes from the instruction after the interrupted instruction. **Take care of corner cases like Branching.**

ISA Specifications

A) Registers

```
R[0:7]<31:0>        ; Eight 32-bit general purpose registers
PC<31:0>            ; 32-bit program counter
SP<31:0>            ; 32-bit stack pointer
CCR<3:0>            ; condition code register
Z<0>:=CCR<0>        ; zero flag, change after arithmetic, logical, or shift operations
N<0>:=CCR<1>        ; negative flag, change after arithmetic, logical, or shift operations
C<0>:=CCR<2>        ; carry flag, change after arithmetic or shift operations.
```

A) Input-Output

```
IN.PORT<31:0>       ; 32-bit data input port
OUT.PORT<31:0>      ; 32-bit data output port
INTR.IN<0>          ; a single, non-maskable interrupt
RESET.IN<0>         ; reset signal
```

Rsrc1 ; 1st operand register
Rsrc2 ; 2nd operand register
Rdst ; Result register field
Offset ; Address offset (16 bit)
Imm ; Immediate Value 16 bits

Take Care that Some instructions will Occupy more than one memory location

Mnemonic	**Function**
One Operand	
NOP	PC ← PC + 1
HLT	Freezes CPU (No Clock Input) until a reset
SETC	C ←1
NOT Rdst	NOT value stored in register Rdst R[Rdst] ← 1's Complement(R[Rdst]); If (1's Complement(R[Rdst]) = 0): Z ←1; else: Z ←0; If (1's Complement(R[Rdst]) < 0): N ←1; else: N ←0
INC Rdst	Increment value stored in Rdst R[Rdst] ←R[Rdst] + 1; If ((R[Rdst] + 1) = 0): Z ←1; else: Z ←0; If ((R[Rdst] + 1) < 0): N ←1; else: N ←0 Updates carry
OUT Rsrc	OUT.PORT ← R[Rsrc]
IN Rdst	R[Rdst] ←IN.PORT
Two Operands	
MOV Rdst, Rsrc	Move value from register Rsrc to register Rdst
SWAP Rdst, Rsrc	Exchange the values between 2 registers
ADD Rdst, Rsrc1, Rsrc2	Add the values stored in registers Rsrc1, Rsrc2 and store the result in Rdst and updates carry If the result =0 then Z ←1; else: Z ←0; If the result <0 then N ←1; else: N ←0
SUB Rdst, Rsrc1, Rsrc2	Subtract the values stored in registers Rsrc1, Rsrc2 and store the result in Rdst and updates carry If the result =0 then Z ←1; else: Z ←0; If the result <0 then N ←1; else: N ←0
AND Rdst, Rsrc1, Rsrc2	AND the values stored in registers Rsrc1, Rsrc2 and store the result in Rdst If the result =0 then Z ←1; else: Z ←0; If the result <0 then N ←1; else: N ←0
IADD Rdst, Rsrc ,Imm	Add the values stored in registers Rsrc to Immediate Value and store the result in Rdst and updates carry If the result =0 then Z ←1; else: Z ←0; If the result <0 then N ←1; else: N ←0
CMP Rdst, Rsrc	Subtract the values stored in registers Rsrc1, Rsrc2 and ignore the result and **updates carry** If the result =0 then Z ←1; else: Z ←0; If the result <0 then N ←1; else: N ←0
SHR Rdst, Imm	Shift [Rdst] right by Imm If the result =0 then Z ←1; else: Z ←0; If the result <0 then N ←1; else: N ←0
SHL Rdst, Imm	Shift [Rdst] left by Imm If the result =0 then Z ←1; else: Z ←0; If the result <0 then N ←1; else: N ←0; Updates carry

Memory Operations	
PUSH Rsrc	X[SP] ← R[Rsrc]; SP-=1
POP Rdst	SP+=1; R[Rdst] ← X[SP];
LDM Rdst, Imm	Load immediate value (16 bit) to register Rdst R[Rdst] ← Imm<15:0>
LDD Rdst, offset(Rsrc)	Load value from memory address Rsrc + offset to register Rdst R[Rdst] ← M[R[Rsrc] + offset];
STD Rsrc1, offset(Rsrc2)	Store value that is in register Rsrc1 to memory location Rsrc2 + offset M[R[Rsrc2] + offset] ←R[Rsrc1];
Branch and Change of Control Operations	
JZ Imm	Jump if zero If (Z=1): PC ←Imm; (Z=0)
JN Imm	Jump if negative If (N=1): PC ←Imm; (N=0)
JC Imm	Jump if carry If (C=1): PC ←Imm; (C=0)
JMP Imm	Jump PC ←Imm
CALL Imm	(X[SP] ← PC + 1; sp-=1; PC ← Imm)
RET	sp+=1, PC ←X[SP]
INT index	X[SP] ← PC + 1; sp-=1;Flags reserved; PC ← M[index + 2] index is either 0 or 1.
RTI	sp+=1; PC ← X[SP]; Flags restored

Input Signals	
Reset	PC ← M[0] //memory location of zero
Interrupt	X[Sp]←PC; sp-=1;PC ← M[1]; Flags preserved

Phase1 Requirement:

- Instruction format of your design
  - Opcode of each instruction
  - Instruction bits details
- Schematic diagram of the processor with data flow details (using a digital tool **not hand written**).
  - ALU / Registers / Memory Blocks
  - Dataflow Interconnections between Blocks & its sizes
  - Control Unit detailed design
- Pipeline stages design
  - Pipeline registers details (Size, Input, Connection, …)
  - Pipeline hazards and your solution including
    i. Data Forwarding
    ii. Static Branch Prediction

Phase2 Requirement

- Implement and integrate your architecture
  - VHDL Implementation of each component of the processor
  - VHDL file that integrates the different components in a single module
- Simulation Test code that reads a program file and executes it on the processor.
  - Setup the simulation wave

    - Load Memory File & Run the test program
- Assembler code that converts assembly program (Text File) into machine code according to your design (Memory File)
- Report that contains any design changes after phase1
- Report that contains pipeline hazards considered and how your design solves it.
- Quartus Flow Summary screenshot of the static-prediction version (and of the final version, if different).
- The memory must be implemented with on-chip RAM blocks (M10K). The Flow Summary's "Total block memory bits" must be non-zero, or the design will not be ranked.

Project Testing

- You will be given different test programs. You are required to compile and load it onto the RAM and **reset** your processor to start executing from the memory location written in address 0000h (PC = M[0]). Each program would test some instructions (you should notify the TA if you haven't implemented or have logical errors concerning some of the instruction set).
- You MUST prepare a waveform using do files with the main signals showing that your processor is working correctly (R0-R7, PC, SP, Flags, CLK, Reset, Interrupt, IN.port, Out.port). Show main signals and only main signals please, try to keep wave clean.
- You need to make sure that your program can generate the programming file.
- Compile using Quartus Prime Lite Edition 16.1 (version 16.1.0.196) with the Cyclone V device support installed, targeting 5CSEMA5F31C6, with **default compiler settings**. Show the Compilation Report → Flow Summary. The TA will record ALMs and total registers from it.

Evaluation Criteria

- Each project will be evaluated according to the number of instructions that are implemented, and Pipelining hazards handled in the design. Table 2 shows the evaluation criteria in detail.
- Failing to implement a working processor will nullify your project grade. No credits will be given to individual modules or a non-working processor.
- Unnecessary latching or very poor understanding of underlying hardware will be penalized.
- Individual Members of the same team can have different grades, you can get a zero grade if you didn't work while the rest of the team can get fullmark, Make sure you balance your Work distribution.
- Working teams will be ranked by FPGA resource utilization, as reported by Quartus after a full compilation targeting the DE1-SoC (Cyclone V, 5CSEMA5F31C6). Ranking is based on **Logic utilization (in ALMs)**; ties are broken by total registers.
- **Only designs that pass all test programs are ranked.** The ranking is based on the version of the design with static branch prediction only. Teams that implement the dynamic prediction bonus must be able to disable the predictor (using a VHDL generic or constant) and compile this static version for the ranking. Both versions (predictor on and off) must pass all test programs.
- Memory bits and pin count are fixed by the spec and are not part of the ranking.

Table 2: Evaluation Criteria

Marks Distribution	Instructions	12 marks
	Resource Utilization Ranking	5 marks
	Data Hazards	1 mark
	Structural Hazards	1 mark
	Control Hazards	1 mark
Bonus Marks	2-bit dynamic branch prediction with address calculation in fetch	2 marks bonus

Team Members

- Each team shall consist of a maximum of four members

Phase 1 Due Date

- Delivery a softcopy on google classroom.
- Week 11, the discussion will be during the regular lab session.

Project Due Date

- Delivery a softcopy on google classroom.
- Week 13, the demo will be during the regular lab session.

General Advice

1. Compile your design on regular bases (after each modification) so that you can figure out new errors early. Accumulated errors are harder to track.
2. Start by finishing a working processor that does all one operands only. Integrating early will help you find a lot of errors. You can then add each type of instructions and integrate them into the working processor.
3. Use the engineering sense to back trace the error source.
4. As much as you can, don't ignore warnings.
5. Read the transcript window messages in Modelsim carefully.
6. After each major step, and if you have a working processor, save the design before you modify it (use a versioning tool if you can as git & svn).
7. Always save the ram files to easily export and import them.
8. Start early and give yourself enough time for testing.
9. Integrate your components incrementally (i.e: Integrate the RAM with the Registers, then integrate with them the ALU …).
10. Use coding conventions to know each signal functionality easily.
11. Try to simulate your control signals sequence for an instruction (i.e: Add) to know if your timing design is correct.
12. There is no problem in changing the design after phase1, but justify your changes.
13. Always reset all components at the start of the simulation.
14. Don't leave any input signal float "U", set it with 0 or 1.
15. Remember that your VHDL code is a HW system (logic gates, Flipflops and wires).
16. Use Do files instead of re-forcing all inputs each time.
17. Check the Flow Summary regularly while developing, not only at the end; warnings about inferred latches usually point to wasted area.